# NetPractice

***Routing Error? More Like User Error.***

*Written By: Luis "Mapache" Torcate*

A set of practical exercises designed to provide an introduction to the basics of computer networking. All about IP addresses, Routing, Subnet Masks and what makes a Network go *brrr*.

# Preamble

For this project, as part of our core curriculum. The 42 school provides us with a training interface that contains several networking problems with increasing difficulty. This suite comes in the form of a group of files that we need to extract.

One of these files is an executable shell script that will launch a web server and open our default web browser to the dedicated page.

Due to technical design and security constraints on various web browsers, it is required to use a local web server to deliver NetPractice's web pages. If the "run.sh" script does not function properly, we are asked to access the project manually.

This is accomplished by running "python3 -m http.server *PORT*" (adding any port number), and then navigating to the URL "http://localhost:*PORT*". The *.http* files included will then allow us to run and interact with the training interface.

We then can practice by entering our intranet login on the "training" tab. Or use the "evaluation" tab to generate a random configuration, also suitable for evaluations.

For each level, a non-functioning network diagram is displayed. At the top of our window, we have one or more objectives that we must achieve by adjusting the available configuration so that the network functions properly.

Then there are two buttons we can use:

- [**Check again**] to verify whether our configuration is correct.
- [**Get my config**] to download our configuration whenever we need to. (This creates a .json file that we will need for submitting our assignment.

Once the level is completed successfully. A new button will appear. Allowing us to progress to the next level. During the evaluation, we must be able to complete three random exercises like these in under 15 minutes.

To complete this assignment, it is necessary to understand how addressing works in a network that includes devices such as routers and switches. The following document is the result of my research notes, and conclusions, regarding IPv4 network administration, routing, subnetting, and TCP/IP configuration. All in preparation to complete the project and its subsequent evaluation.

*Luis "Mapache" Torcate*

# IP Addresses

Internet Protocol Address (IP Address) is a 32-bit number, often depicted as 4 groups of numbers separated by dots that serve to determine the “where” of a machine, be it a client, a router or a server. If you have a house, it is the address of that house. Looks something like:

**192.168.2.10**

This address is composed of two sections, a ***Network*** part and a ***Host*** part. Following the house analogy, your house, can be a part of a larger neighborhood. So the **Network** could be something like the postal code of that neighborhood. Then the information specific to your house, like the name of the street, house number, would be the **Host** section.

That said, the Host and Network sides can be within one of those numbers that’s between 2 dots, because each of those values is a human-readable way of looking at the 8-bit arrangement that’s on each of those groups.

To understand where the Network side ends and the Host begins, we need to check the “*locks*” from this address. What is locked, is the **Network** info, and what is not, is your **Host** info. To consult these “*locks*”, we make use of the Networks’s ***Subnet Mask***.

*Luis “Mapache” Torcate*

## Subnet Masks

These Masks are also 4 groups of numbers separated by dots. A directly aligned 32-bit arrangement that “locks” each slot bit-by-bit. Working to indicate whether this “slot” belongs to the Network address or not. Following with the example above, that IP address (192.168.2.10), could have a subnet such as:

**255.255.255.0**

This would indicate that the first 3 groups, showing “255”, belong to the Network address. Unlike the last one, showing “0”. Meaning that the “10” from the example address refers to the Host part of the address.

Because the “locks” are set per-bit, a value between 2 dots can be partially of the Network and partially of the Host.

These “locks” by the way, is a metaphor that helps visualize how the label is structured. Is not really setting permissions or something of the sorts.

If a packet is being sent, the IP address of the sender and receiver are contrasted against the Networks’s subnet Mask. If these labeling “locks” determines that there’s a match at the sections comprehending the Network’s address, then the package is sent with a simple switch.

Otherwise, it is sent to the ***Router*** to be sent to a different network. At this level, these “locks” actually just make a sort of “rule” for the Network. Establishing which bits of the addresses of the machines connected to that ***Router*** will correspond to either of them, identifying different networks at the moment of sending packages.

That said, these masks are always established from left to right. So the “locks” will never be mixed, and this guarantees that the information of the address regarding the Network, will always be the left-most bits. This fact gave birth to a different type of notation for these masks.

The **CIDR** notation (*Classless Inter-Domain Routing*) shows a forwards-slash followed by a value that represents how many of the starting bites comprehend the Network address (/X). So using the example from before (*255.255.255.0*) since each of the groups is an **octet** (an 8-bit value), we would use the **CIDR** notation /24. This is because 255 would be 11111111 in binary, so that’s eight 1’s indicating that the first group represents solely the Network. Multiply this by the 3 groups that show the same, and you get the 24 from the /24 **CIDR** notation.

As a note, just for clarity, a network can have multiple IPs that connect to different machines. But it will always have only **one** subnet mask.

*Luis “Mapache” Torcate*

## Sending a Package

Picture a situation where you want to send a package. First, your OS will use its subnet to contrast it with the IP that you are sending the package to, to verify if it is within the same network. If it is, it will send it accordingly.

If it isn't, it will direct it to the ***Router*** that's set as the "**Default Gateway**". This Router will then internally contrast the IP with its own Routing Table. The Routing Table has access to subnets, and therefore, part of the IP of all the networks and/or ***Routers*** connected to it.

For example, in an Office building, there can be several networks dedicated to different departments. If a machine in HR, needs to send a package to Payroll, the package will go to the Office's internal Router, and this one will send it there.

If the IP where the package is being sent, does not match with any of the Router's routing table, it will send the package to your main **ISP** router. **ISP** stands for *Internet service provider,* and here is where everything goes if your router doesn't find a place where to send your package.

The IP for the **ISP** is irrelevant, at this point. This is because the router will simply have a ***Default Route*** set like: 0.0.0.0 /0. Meaning that its subnet mask will accept any packages from any addresses. By default, your Router should be able to dump things here if everything else fails, but this may also be blocked or removed from the table, meaning that you won't be able so send packages outside your Router's routing table of networks, and this scenario would likely return an error.

Now that our package has been delivered, we have to consider that this sending is more a conversation than a monologue, so after all is done, a different "package" is sent back to the sender, which is the confirmation that the package was received successfully, terminating the conversation for that particular package. If a network arrangement is only set in one direction, the message might be sent, but without the confirmation, it will likely shoot out an error

# DNS

DNS stands for Domain Name System, and it serves as a human-readable way to get to different websites. Computers only read 1s and 0s, but the DNS gives us the option to use an easier to remember *Domain Name* to access a specific IP address.

Not to be confused with a URL. The **URL** (Uniform Resource Locator) Is the entire "***path***" to the resource you are locating, and it contains several components. As an example, let's analyze this **URL**:

https://www.wikipedia.org/wiki/Subnet

1. **https://** (*The Protocol*): Tells the computer how to talk securely with the source of the resource.
2. **www.wikipedia.org** (*The Domain Name*): Tells the computer who to talk to. This is the only part DNS cares about.
3. **/wiki/Subnet** (*The Path*): Tells the destination server exactly which file you want.

The domain name is what the DNS actually uses. It will essentially look up the name and return the IP of the website, or resource it finds. To do this, it makes a couple steps:

1. ***The Request:*** You type wikipedia.org into your browser. Your computer doesn't know the IP, so it asks a DNS Resolver (usually provided by your ISP, or a public one like Google's 8.8.8.8).
2. ***The Search:*** If the resolver doesn't know the answer, it climbs a hierarchy of servers:
    a. *Root Servers:* Direct the resolver to the servers handling .org.
    b. *TLD (Top-Level Domain) Servers:* Direct the resolver to the specific servers managing Wikipedia.
    c. *Authoritative Servers:* Hold the exact, official IP address for wikipedia.org.
3. ***The Answer:*** The resolver brings the IP address back to your computer.
4. ***The Connection:*** Your browser uses that IP address to connect directly to the website server.

*Luis "Mapache" Torcate*

# Switches

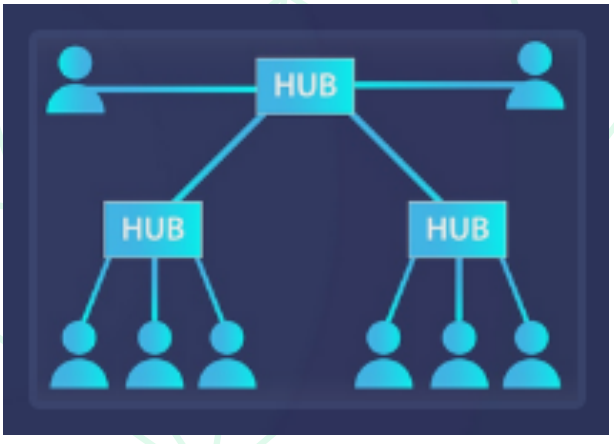
Switches in real life operate at smarter level than the ones in the exercises. They contrast the information in the package with the MAC Address of the machine that is connected to it, in order to know if It's where they should be delivered. The ones on the exercises work more like **Hubs.** These are "*dumb*" elements of the architecture that simply connect multiple machines to a centralized communication point.

But for the sake of simplicity, we'll refer to them as **switches** like the training interface does. A **Switch** doesn't have its own IP, nor does it relay information of the machines plugged into it, the ***Router*** simply will have an IP like 192.180.1.0 /24, on its routing table, and all the machines in the network will be something like 192.180.1.1 /24 192.180.1.3 /24 192.180.1.2 /24 192.180.1.4 /24 and so on.

So if the ***Router*** needs to send a package to any of those machines, it will simply send them to the **Switch** without even knowing if the specific machine is there, and the **Switch** will try all machines connected to it, even after it has found a match.

A **Switch** can also exist without a ***Router***, and this is for situations where machines only need to communicate between each other. Instead of having all machines interconnected with a bunch of cables, all of them can be connected to the same **Switch** to talk with each other.

This also opens the possibility for many ***Routers*** to communicate to each other in a potentially endless tree. Many ***Routers*** can be connected to the same **Switch** to talk to each other, or even that **Switch** can be connected to a different central ***Router*** that's connected to a different switch with the same arrangement and so on. Possibilities are endless.




*Luis "Mapache" Torcate*

## Router to Router

On less complex architectures, you can simply have 2 routers communicate to each other. On those cases, commonly, you'd find a /30 subnet.

On networking, there's a rule, that states that the first and last addresses shouldn't be given to any particular machine.

The first address, is supposed to be ***The Network Address*** it gives the "where" of the Network as a whole, so it shouldn't link to a specific ID, giving options like the one I explained before regarding switches, to work as intended.

The last address, is supposed to be ***The Broadcast Address***. Remember that the IP address is a human-readable way to see what is really a group of 4 binary octets. So for example, in a /24 network, all the IPs for the Hosts are the possible combinations of 1s and 0s inside the last octet. Therefore, if a package is sent to the last address (x.x.x.255 in a /24 network), then all the possible places where a 1 can be, will be. So this address matches all the possible hosts, and gives the opportunity to send a message to all of them at the same time. Hence, the name ***Broadcast Address***.

When a computer sends a packet with the destination set to the ***Broadcast Address***, the network ***Switch*** acts like a megaphone. It instantly duplicates the packet and blasts it out of every single port to every device in the network.

So with this in mind, let's look back at that /30 subnet.

An IP address holds 4 octets like we know, so there are a total of 32 bits to write any address. The /30 rule then, leaves only 2 bits to create host addresses, which gives us only 4 possible combinations:

- 00
- 01
- 11
- 10

So, considering what we talked about just now, about the first and last address having a specific use, there are really only 2 possible addresses, so it makes it perfect to connect 2 routers together, without leaving any excess addresses. Secure, functional and reliable.

If multiple routers needed to be connected to each other, the same system would work. This is because a router has different ports where those same 2 IP networks can be set up, so you may create multiple /30 networks to connect multiple routers. That said, if there are many more, might be better to use a switch, but if it is just 2 or 5, then you are cool making it this way.

*Luis "Mapache" Torcate*

# TCP

Transmission Control Protocol, is a system built on top of the information delivery system that we've explained so far. The IP by itself will solely manage that the package goes to where it needs to, but on its own, the package could come broken, corrupted or could straight up be missing information.

This protocol establishes a connection between the sender and the receiver, and checks on every package of information if it was received. This is accomplished by different signals.

1. The Firsts signal is the ***SYN*** (*Synchronize*), It is sent to establish the connection. The sender emits it and waits for response.

2. The Second signal is the ***SYN-ACK*** (*Synchronize Acknowledge*). This one is sent back from the receiver confirming that the connection has been established. The receiver sends this signals and waits for the final signal of the protocol.

3. The Third signal is the ***ACK*** (*Acknowledge*). This one is sent from the sender after it receives the SYN-ACK back from the receiver. This one once it reaches the receiver lets it know that the data is about to start coming through.

Then, for every data package received, the receiver will send an ***ACK.*** This one is sent back confirming that every package is received successfully. This is where the magic happens.

If the ACK is not received for a given package, it will send a copy of the package until it is, or times out. Also, each package will have a ***Sequence Number,*** that will be used to ensure that the information is not disordered at the end. This combined gives opportunities for all the data to be sent successfully, and ensures that it will be delivered correctly.

The protocol does not send these packages one by one though. That would take a long time. In reality is more of a stream of packages, and eventually it will confirm if it received the corresponding ACK for all the packages it sent. This is called the **TCP Sliding Window.** That said, this is still kinda slow for some purposes.

If you want to send an image or something complex, it is ok for it to take a bit. But what about the frame rate of a video game or a video conference? For these cases where over a thousand images need to be sent every minute, a different system is used. Like **UDP** (User Datagram Protocol), which just throws the data as fast as possible and doesn't really care about the state of the package, or if it even arrives. Kinda like AliExpress.

As a fun fact, the word *package* is often linked to ***TCP***. For ***UDP*** the same thing is often referred to as a *datagram*.

*Luis "Mapache" Torcate*

# (MIME) Multiple Interface Match Error

MIME is a mnemonic device I created to never forget the devil that lives in this project.

I had this issue a couple of times and I used to solve it just by trying different addresses until it worked. But there's a better way of fixing this error. Understanding it.

I was under the impression that an IP address gave the router the exact directions to the machine, but I was mistaken. In reality, the router sends the data to a **range** based on that address.

If you think of it like a post office, is like the router doesn't send an individual mailman a letter for a specific house and sends it on its way. In reality, Mr. Postman, puts all letters that go to a general ***route*** into a truck, and that truck is then sent on its ***Route***. So if the specific address is a part of said route, it will reach it, but this opens up the possibility of a MIME messing up your routing.

See, the *subnet mask* of a network will give you the range of IPs that your host machines can have, this part should be clear by now. If you have a /30 mask, take away the network address and the broadcast address, and you are left with only 2 possible IPs. So far so good.

That said, within this number of addresses, there is a particular segmentation that the router "sees" when sending the packages to the machines. These ***ranges*** of IPs is what would be the different ***routes*** for the truck in the previous example.

The **MIME** comes up when you forget about this. Or just don't know it exists like I used to. For example, if you had an /30 network like 10.0.0.8, we know that the available host IPs are .9 and .10. That said, if you then had another network connected to that router, that was a /24, and you link it up with a machine like 10.0.0.3, you'll get a **MIME** putting up a wall that you won't get passed.

This is because those addresses are within the same "**Route**" or "**range**", so when the router needs to send a package to either of those networks, even if the networks and the IPs are clearly different, it will have **Multiple Interfaces Matching** the general "**Route**" or "**range**" for the package, and it will shoot an error.

*Luis "Mapache" Torcate*

We can avoid the **MIMEs** keeping this in mind:

- Each "**range**" within a set of IP addresses, has a fixed number of addresses that can be defined as 2 elevated to the power of as many bits are left for the **Host** machines IPs
    - So in a network with a /25 subnet, there are 7 bits left (32 - 25). So we know that each "**range**" contains 128 addresses. Of which 126 are usable machine IPs.

- All IPs have as many possible addresses, as there are numbers between the all off (*network address*) and all on (*broadcast address*) bit positions.
    - So in a network with a /29 subnet, where the final octet is .156. Consider this, we know that this number (156) reads as 10011**100**, and because of the subnet, we know that the first 5 bits belong to the network, which tells us that the last three (**100**) is the part that will form the individual machine IPs.

      If we switch off all these bits we get the number 152 (10011**000**), and if we turn them on we get 159 (10011**111**). So because of this we know that the IPs go from 152 to 159, meaning that there are only 8 ((159 - 152) + 1) possible IPs on the X.X.X.156 /29 network.

- The number of sections in a set of IP addresses, equals as many "**ranges**" as can fit, within the octets that are either completely or partially untouched by the mask.
    - So in a network with a /29 subnet, only the last octet is partially untouched by the mask, meaning that the number of slices in our pie is 256 / (2^3) or 32. We know that our slice of the pie, or our "range" will have 8 IPs, so if the final octet was .78, since 8*9=72 and 8*10=80, we know that our IP is on the 9th "**range**", and everything within those numbers will be considered a part of its "**route**".

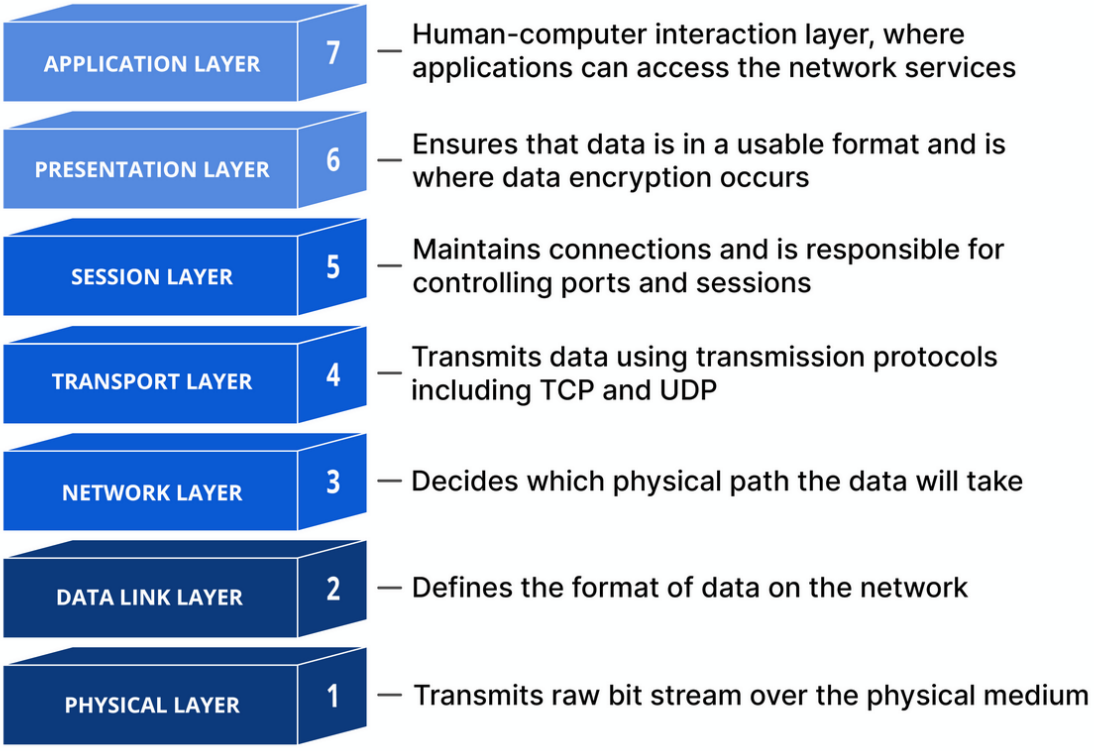
Defining where a segment starts and ends can help us make sure that 2 machines do not share **segments**, so continuing the example from the beginning of the page, we now know that the IPs .9 and .10 were **MIME-ing** with 10.0.0.3 because. Since it has a subnet of /24, it means that it has only one segment, which includes the .9 and .10 addresses from the other network. Meaning that when the router wanted to send data to that route, it didn't know which interface to choose, causing the error. Now we can hate **MIMEs** together. 🦝

*Luis "Mapache" Torcate*

# OSI Layers

This is a protocol created by the *International Standards Organization* (*ISO*), that defines the different “layers” that compose the communication between systems. In this case, it is being described as it applies to Computer Networks. However, it may also be applied in a number of ambits.

That said, this protocol was later simplified and overtaken by the **TCP/IP** protocol as described in the present document. Nonetheless, below is a diagram that shows it in better detail as there are further aspects that the ***TCP/IP*** protocol doesn’t cover.




*Luis “Mapache” Torcate*

## Special IPs

| The IP Block (Network) | The Name | The Hard-Coded Rule |
|---|---|---|
| **127.0.0.0 /8**<br><br>*(127.0.0.0 to 127.255.255.255)* | **Loopback (Localhost)** | **The "Mirror" Rule:** Traffic sent here never touches a network cable. The OS loops it straight back to its own internal memory. Used for testing software on your own machine. |
| **169.254.0.0 /16**<br><br>*(169.254.0.0 to 169.254.255.255)* | **APIPA (Link-Local)** | **The "Panic" Rule:** If a computer turns on, asks for an IP address, and nobody answers, it assigns itself one of these. **Routers are strictly forbidden from passing these packets.** They only work inside a single switch. |
| **255.255.255.255 /32** | **Global Broadcast** | **The "Local Megaphone" Rule:** Shouts to every single device on your physical network. **Routers will absolutely drop this packet** so you don't accidentally shout at the entire global internet. |
| **0.0.0.0 /0** | **Default Route** | **The "Everything Else" Rule:** Used in Routing Tables to mean *"If you don't have a specific map for the destination, throw the packet out this door."* |

| Private Block | CIDR | How many IPs it holds | Where it's normally used |
|---|---|---|---|
| **10.0.0.0** | /8 | ~16.7 Million | Massive enterprise companies, huge data centers, and NetPractice router links. |
| **172.16.0.0** to<br><br>**172.31.255.255** | /12 | ~1 Million | Medium-sized office buildings and college campuses. |
| **192.168.0.0** | /16 | ~65,000 | Almost every home WiFi network and small office in the world. |

*Luis “Mapache” Torcate*

## Practical Conclusions and Evaluation Tips

1. If you have two sets of machines connected to two routers that need to maintain similar IPs, a good rule of thumb seems to be to have the last octet of one group begin with a value above 128 and the other one with a value below it. This ensures that the first bit of that last octet will be 1 for one group and 0 for the other, making easier the distinction with a mask of /25 or above.

2. When sending a package back from the ISP, you have to set the IP and mask that will match all the machines that you will have connect to the internet. This will influence also the set of IPs that you can use for these machines, and depending on your configuration, the previous tip could be essential. Just keep in mind that in that case you'll want a /24 mask or less of course.

3. If you have a /30 network in your configuration, try to make it the lowest address possible. In my experience, makes the mental math for other IPs connected to that router easier. This is specifically related to avoiding MIMEs, and will ensure that you simply need to avoid the first range of IP addresses.

4. The **bc** in-terminal calculator often glitches when switching input and output bases to calculate values in binary. Might be worth opening several instances for different purposes, in order to save time and avoid confusion with unexpected results. I've found that 3 instances is enough.

5. Learning to read the error log section is key to complete this project. Instead of doing the exercises multiple times until you can solve them by memory, take the time to understand each error, its root and potential solutions. This will give you the edge you need to avoid getting stuck in an evaluation, and it is also a better practical skill to develop.

*Luis "Mapache" Torcate*